

Reinforcement Learning

Dr. Pejman Ebrahimi



UNIVERSITÄT
LIECHTENSTEIN

Deep Learning and Advanced AI Techniques
Course

Table of contents

01

Introduction

02

Q-Learning

03

RL Games

04

Proximal Policy
Optimization (PPO)

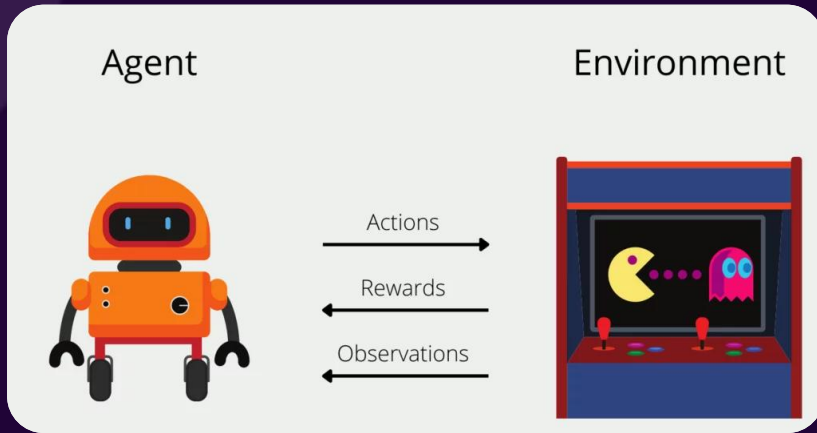
05

RLHF



01 Introduction

What is Reinforcement Learning?



- Type of machine learning where agents learn by interacting with an environment
- Agent performs actions and receives feedback through rewards
- Learning happens through trial and error
- Goal: Learn optimal behavior (policy) that maximizes total reward
- No direct supervision - only reward signals

Key Components of Reinforcement Learning

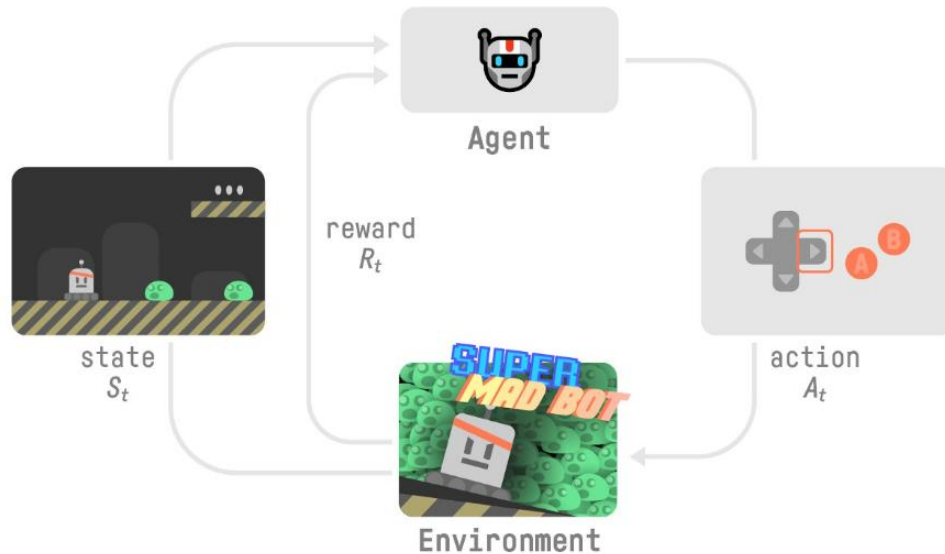
- ❑ **Agent:** The learner or decision-maker
- ❑ **Environment:** Everything the agent interacts with
- ❑ **State (s):** Current situation of the agent
- ❑ **Action (a):** Choices the agent can make
- ❑ **Reward (r):** Feedback signal (positive or negative)
- ❑ **Policy (π):** Agent's strategy for selecting actions
- ❑ **Learning Rate (α):** How quickly the agent incorporates new information
- ❑ **Discount Factor (γ):** How much future rewards matter

Simple Example

- imagine putting your little sister or brother in front of a video game he never played, giving him a controller, and leaving him alone. Your sister/brother will interact with the environment (the video game) by pressing the right, left, up and down buttons (action) to finally learn how to play based on positive and negative scores.



RL Process

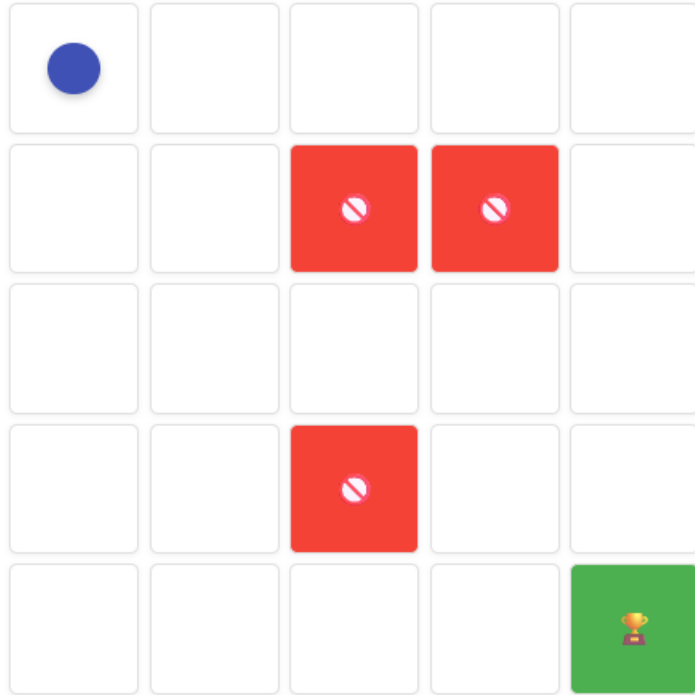


- Our Agent receives **state** S_0 from the **Environment** — we receive the first frame of our game (Environment).
- Based on that **state** S_0 , the Agent takes **action** A_0 — our Agent will move to the right.
- The environment goes to a **new state** S_1 — new frame.
- The environment gives some **reward** R_1 to the Agent — we're not dead (Positive Reward +1).

Real-World Examples

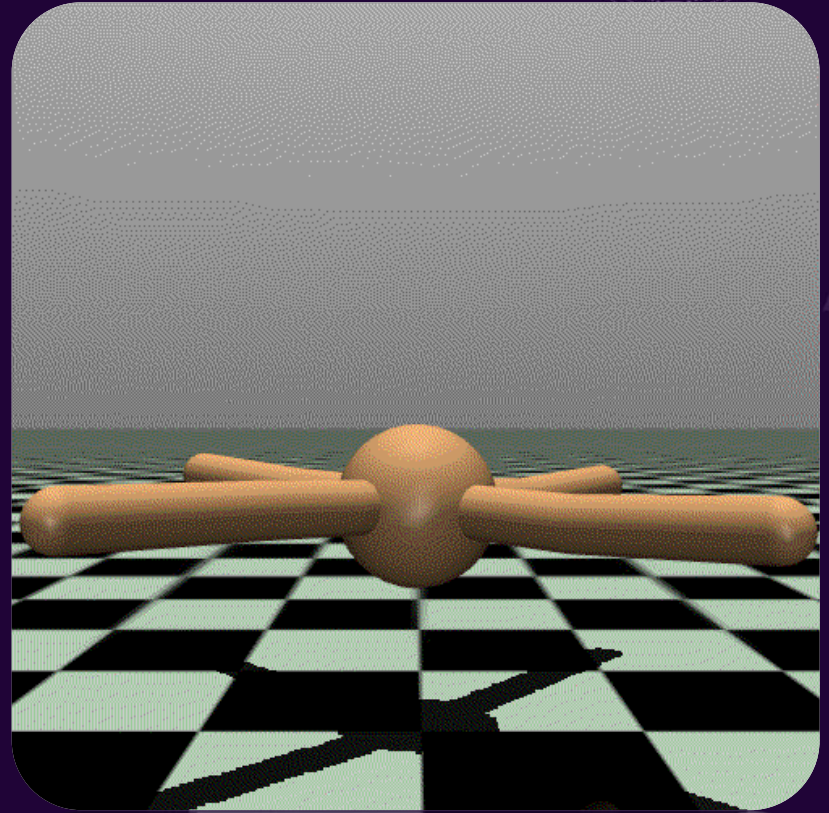
- ❑ AlphaGo/AlphaZero mastering complex games
- ❑ Robotics learning to walk or manipulate objects
- ❑ Recommendation systems personalizing content
- ❑ Resource management (data centers, power grids)
- ❑ Autonomous vehicles learning driving behaviors
- ❑ Trading systems optimizing investment strategies

Grid World



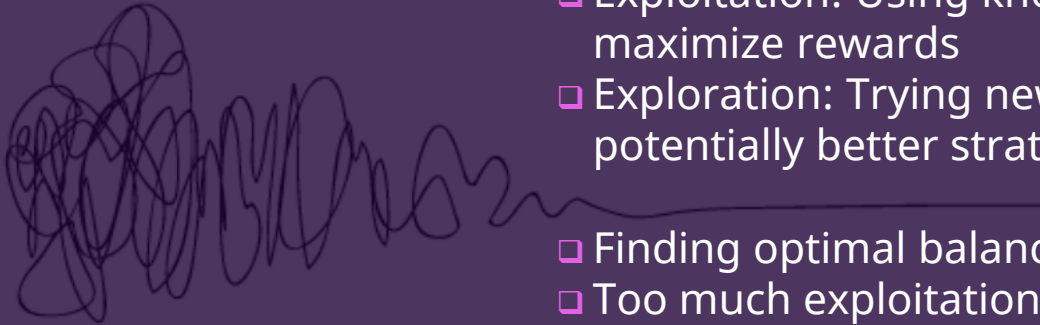
■ Agent ■ Goal (+10) ■ Obstacle (-5) ■ Path

■ Agent ■ Goal (+10) ■ Obstacle (-5) ■ Path



Exploration vs. Exploitation

Explore



Higher uncertainty

Exploit

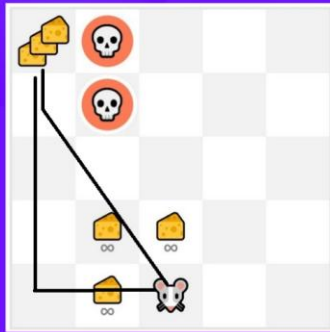
- ❑ Exploitation: Using known information to maximize rewards
- ❑ Exploration: Trying new actions to discover potentially better strategies
- ❑ Finding optimal balance is crucial:
- ❑ Too much exploitation → miss optimal solutions
- ❑ Too much exploration → inefficient learning, excessive costs

Lower uncertainty

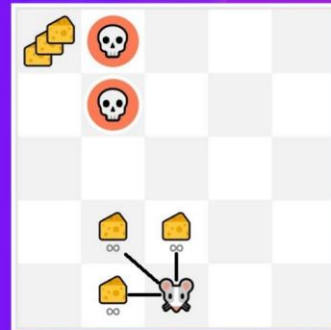
Exploration vs. Exploitation

Exploration/ Exploitation tradeoff

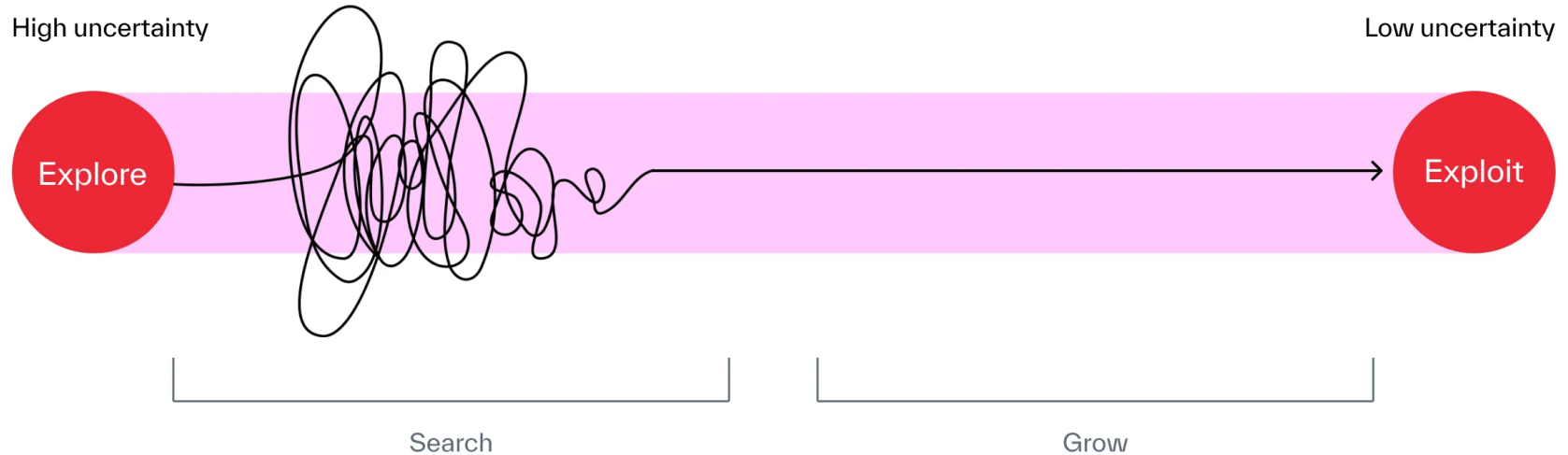
Exploration: trying random actions in order to find more information about the environment.



Exploitation: using known information to maximize the reward.



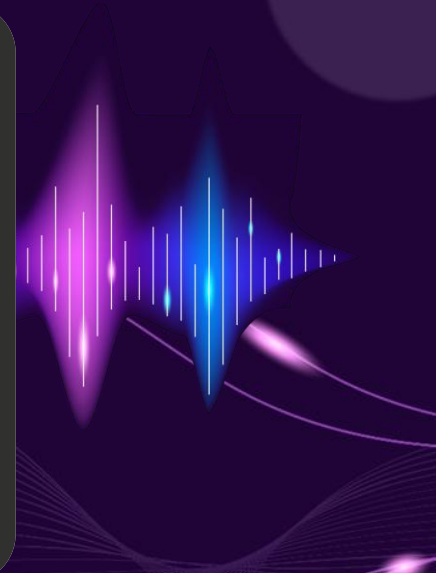
Exploration vs. Exploitation



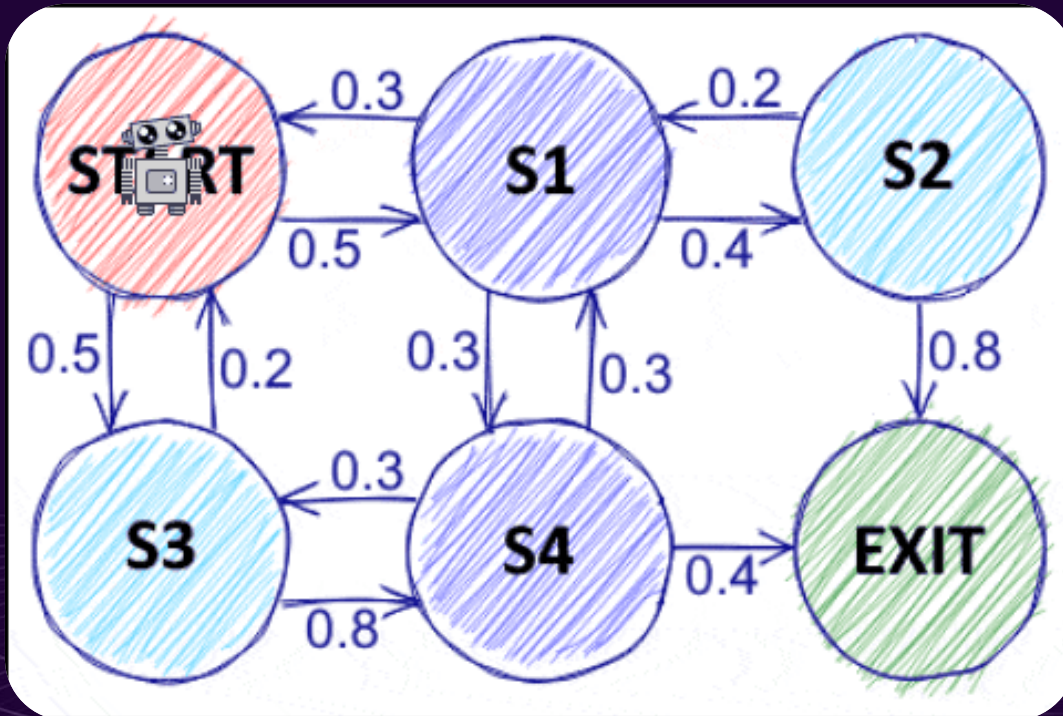
Markov Decision Process (MDP)

the Markov Property implies that our agent needs only the current state to decide what action to take and not the history of all the states and actions they took before

- Mathematical framework for modeling RL problems
- Components: states, actions, transition probabilities, rewards
- **Markov Property:** Future depends only on present state, not history
- Formalized as tuple (S, A, P, R, γ)
 - S : Set of states
 - A : Set of actions
 - P : Transition probability function
 - R : Reward function
 - γ : Discount factor



Markov Decision Processes and Bellman Equations



□ **Markov decision processes** – A Markov decision process (MDP) is a 5-tuple $(S, \mathcal{A}, \{P_{sa}\}, \gamma, R)$ where:

- S is the set of states
- $\mathcal{A}$ is the set of actions
- $\{P_{sa}\}$ are the state transition probabilities for $s \in S$ and $a \in \mathcal{A}$
- $\gamma \in [0, 1[$ is the discount factor
- $R : S \times \mathcal{A} \rightarrow \mathbb{R}$ or $R : S \rightarrow \mathbb{R}$ is the reward function that the algorithm wants to maximize

□ **Policy** – A policy π is a function $\pi : S \rightarrow \mathcal{A}$ that maps states to actions.

Remark: we say that we execute a given policy π if given a state s we take the action $a = \pi(s)$.

□ **Value function** – For a given policy π and a given state s , we define the value function V^π as follows:

$$V^\pi(s) = E \left[R(s_0) + \gamma R(s_1) + \gamma^2 R(s_2) + \dots | s_0 = s, \pi \right]$$

□ **Bellman equation** – The optimal Bellman equations characterizes the value function V^{π^*} of the optimal policy π^* :

$$V^{\pi^*}(s) = R(s) + \max_{a \in \mathcal{A}} \gamma \sum_{s' \in S} P_{sa}(s') V^{\pi^*}(s')$$

Remark: we note that the optimal policy π^ for a given state s is such that:*

$$\pi^*(s) = \operatorname{argmax}_{a \in \mathcal{A}} \sum_{s' \in S} P_{sa}(s') V^*(s')$$

Observations/States Space

Observations/States are the information our agent gets from the environment.



Rewards and the discounting

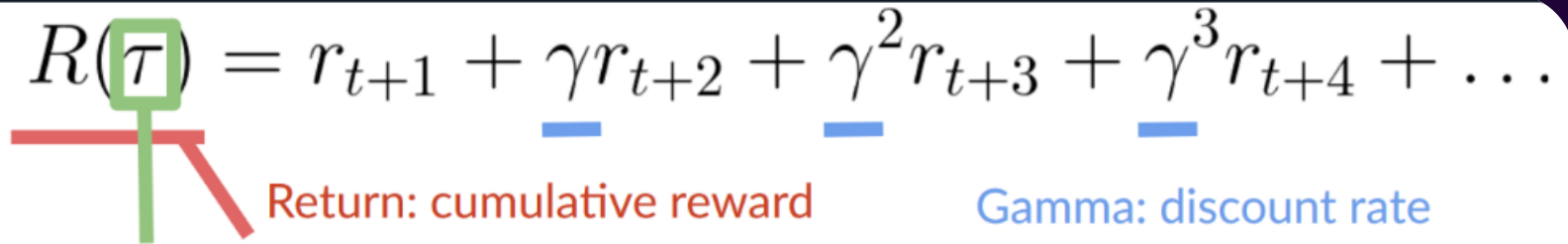
$$R(\tau) = r_{t+1} + r_{t+2} + r_{t+3} + r_{t+4} + \dots$$



Return: cumulative reward

Trajectory (read Tau)

Sequence of states and actions

$$R(\tau) = r_{t+1} + \gamma r_{t+2} + \gamma^2 r_{t+3} + \gamma^3 r_{t+4} + \dots$$


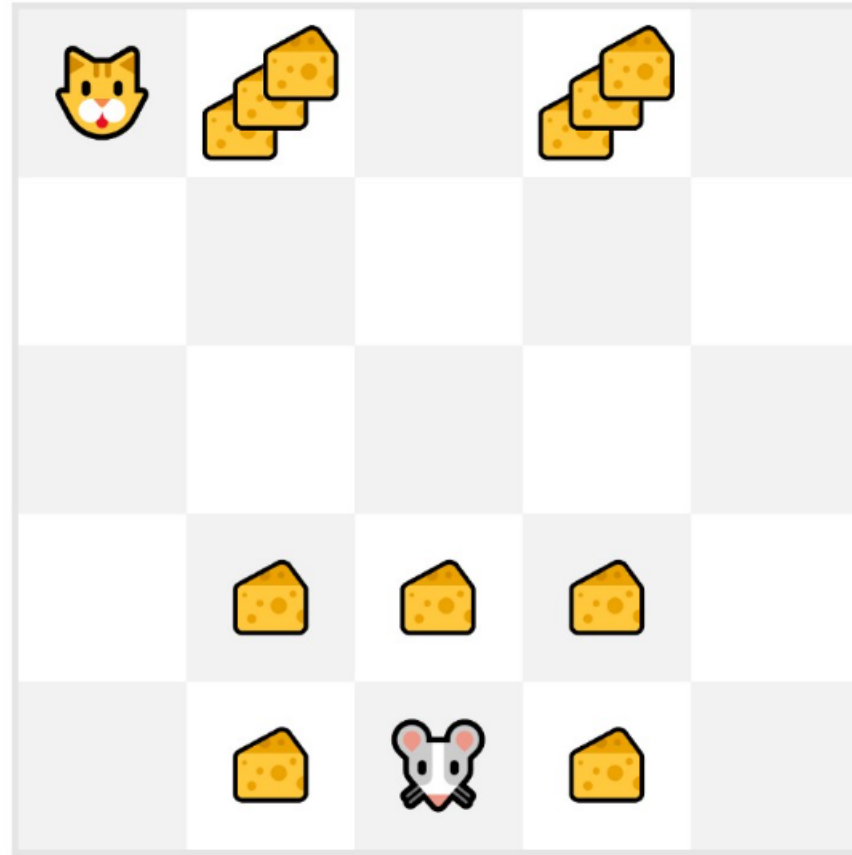
Return: cumulative reward

Gamma: discount rate

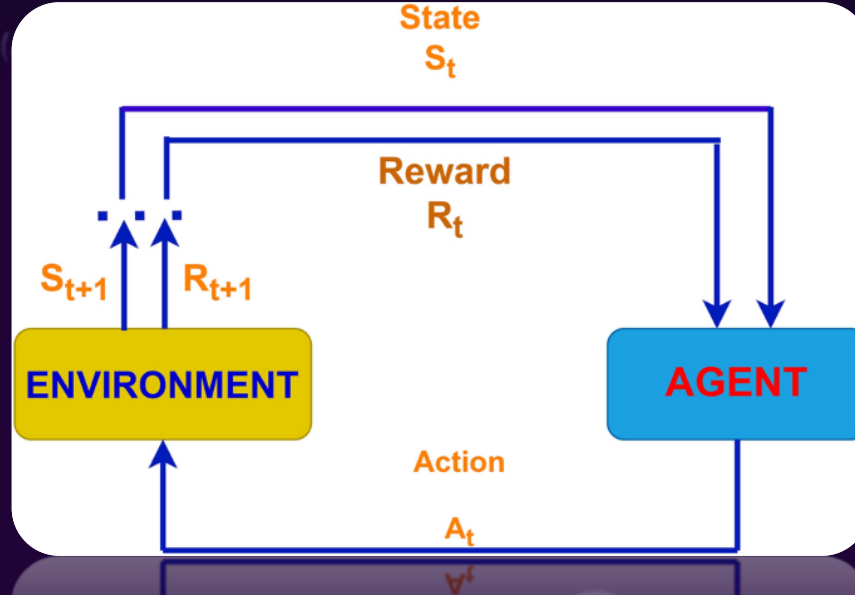
Trajectory (read Tau)

Sequence of states and actions

$$R(\tau) = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1}$$



1. We define a discount rate called gamma. **It must be between 0 and 1.** Most of the time between **0.95 and 0.99.**
 - The larger the gamma, the smaller the discount. This means our agent **cares more about the long-term reward.**
 - On the other hand, the smaller the gamma, the bigger the discount. This means our **agent cares more about the short term reward (the nearest cheese).**



Task



Topic:

Three Main Approaches to Solving Reinforcement Learning Problems
(Value-based vs. Policy-based vs. Actor-Critic)

Instructions:

- You will give a **10–15 minute presentation** to the class on the **three core RL approaches**. Your goal is to **clearly explain** how each method works, their key differences, and where each is best applied.



02

Q-Learning

Q-Learning

- ❑ Foundational reinforcement learning algorithm developed by Chris Watkins (1989)
- ❑ Model-free approach that learns directly from interactions with environment
- ❑ "Q" refers to the quality of actions in states (how valuable each action is)
- ❑ Goal: Learn optimal action-selection policy to maximize cumulative reward

How Q-Value work? (1)

Q-Values

0.00	0.00	0.00	0.00	0.00
0.00 0.00	0.00 0.00	0.00 0.00	0.00 0.00	0.00 0.00
0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00
0.00 0.00	0.00 0.00	0.00 0.00	0.00 0.00	0.00 0.00
0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00
0.00 0.00	0.00 0.00	0.00 0.00	0.00 0.00	0.00 0.00
0.00	0.00	0.00	0.00	0.00
0.00	0.00	0.00	0.00	0.00
0.00 0.00	0.00 0.00	0.00 0.00	0.00 0.00	0.00 0.00
0.00	0.00	0.00	0.00	0.00

History

Performance Metrics

Episodes

0

Current Steps

0

Success Rate

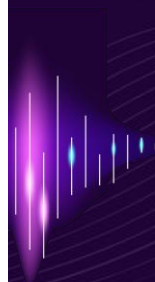
0%

Avg. Steps

0

Learning Progress

■ Steps per Episode



How Q-Value work? (2)

Q-Values

-0.36 -0.36 -0.34	-0.32 4.2 4.84	-0.25 -0.80 -1.30	-0.30 -0.29 -4.64	-0.21 0.35 2.21
-0.32 -0.32 -0.29	3.61 -0.26 5.50	0.00 0.00 0.00	0.00 0.00 0.00	-0.13 2.10 3.81
-0.31 -0.24 -0.24	-0.23 4.5 3.40	-4.50 -0.16 -2.24	-1.30 -0.03 7.91	-0.13 -0.11 -0.05
-0.27 -0.19 -0.28	5.22 -0.25 -0.24	0.00 0.00 0.00	-0.03 8.7 8.90	-0.11 1.00 9.99
-0.25 -0.27 -0.30	-0.25 -0.30 -0.20	-1.31 -0.26 -0.18	0.00 10.0 0.00	0.00 0.00 0.00

History

Performance Metrics

Episodes

20

Current Steps

7

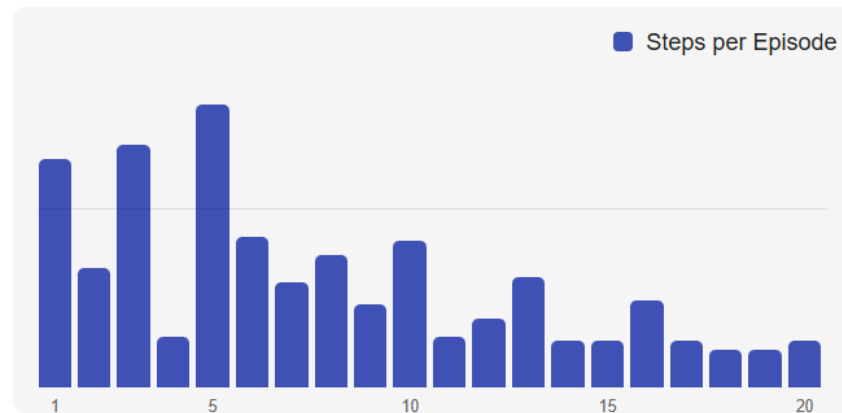
Success Rate

100%

Avg. Steps

23

Learning Progress



How Q-Value work? (3)

Q-Values

3.16 ◀.65 4.2▶ 3.62	3.62 ◀.68 3.5▶ 4.85	-0.25 ◀.25 -0.29▶ -1.30	-0.30 ◀.28 0.35▶ -4.64	-0.21 ◀.27 -0.23▶ 2.21
-0.32 ◀.62 4.8▶ 3.40	4.17 ◀.26 -0.79▶ 5.50	0.00 ◀.00 0.00▶ 0.00	0.00 ◀.00 0.00▶ 0.00	-0.13 ◀.30 2.1▶ 3.81
4.09 ◀.24 4.5▶ -0.26	-0.23 ◀.91 6.2▶ 4.81	-0.21 ◀.38 7.0▶ 1.87	1.26 ◀.34 5.2▶ 7.91	-0.13 ◀.90 -0.11▶ 7.65
-0.27 ◀.26 3.9▶ -0.28	5.49 ◀.25 -0.71▶ -0.24	0.00 ◀.00 0.00▶ 0.00	7.02 ◀.98 8.9▶ 8.90	4.66 ◀.00 1.0▶ 10.00
-0.25 ◀.27 -0.26▶ -0.30	-0.25 ◀.30 6.6▶ -0.20	-1.31 ◀.26 8.9▶ -0.18	7.89 ◀.83 10.0▶ 0.00	0.00 ◀.00 0.00▶ 0.00

Performance Metrics

Episodes

48

Current Steps

2

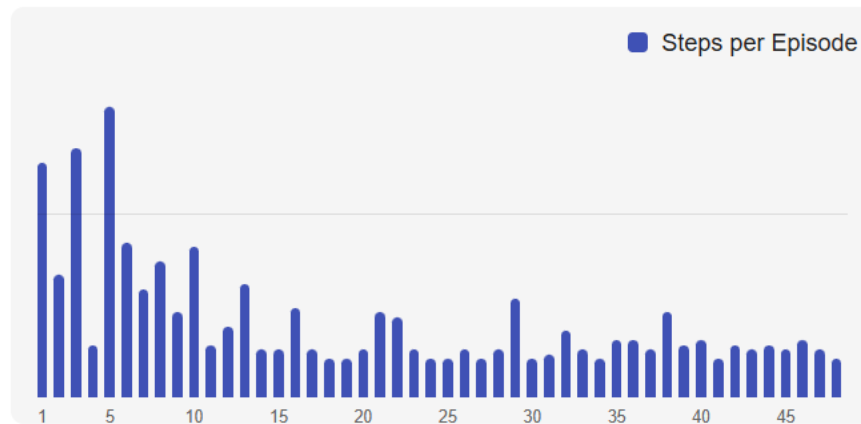
Success Rate

100%

Avg. Steps

16

Learning Progress



History

How Q-Value work? (4)

Q-Values

0.16 0.42 4.2 0.37	1.83 1.44 0.14 4.85	-0.14 1.77 -0.14 -1.72	-0.11 -0.11 -0.11 -2.35	-0.07 -0.09 -0.07 0.01
-0.17 -0.16 -0.13 2.30	1.54 0.28 -3.10 5.50	0.00 0.00 0.00 0.00	0.00 0.00 0.00 0.00	-0.07 -1.36 -0.04 0.60
-0.10 -0.09 4.7 -0.10	1.53 1.13 0.05 6.22	-0.95 -0.07 0.85 -1.36	-0.95 -0.04 2.93 -0.03	0.00 -0.02 0.56 6.53
-0.06 -0.06 6.2 -0.06	1.86 1.63 -2.65 7.02	0.00 0.00 0.00 0.00	-0.00 -0.50 8.7 -0.02	0.73 0.38 1.17 10.00
0.11 0.06 2.7 -0.04	6.18 0.48 7.9 1.86	-2.23 1.14 8.9 3.95	6.51 1.37 10.0 4.66	0.00 0.00 0.00 0.00

Performance Metrics

Episodes

446

Current Steps

6

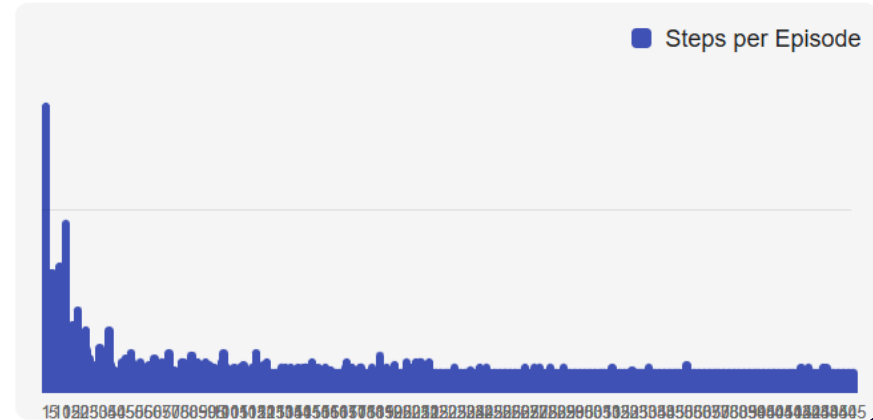
Success Rate

100%

Avg. Steps

10

Learning Progress



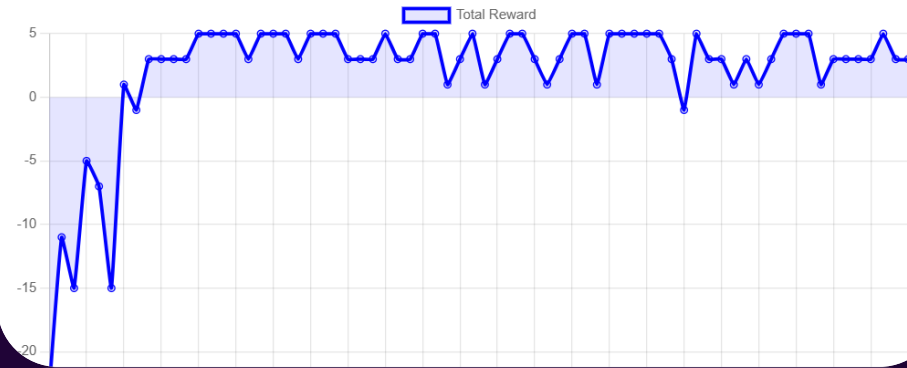
History

Q-Table Simulation

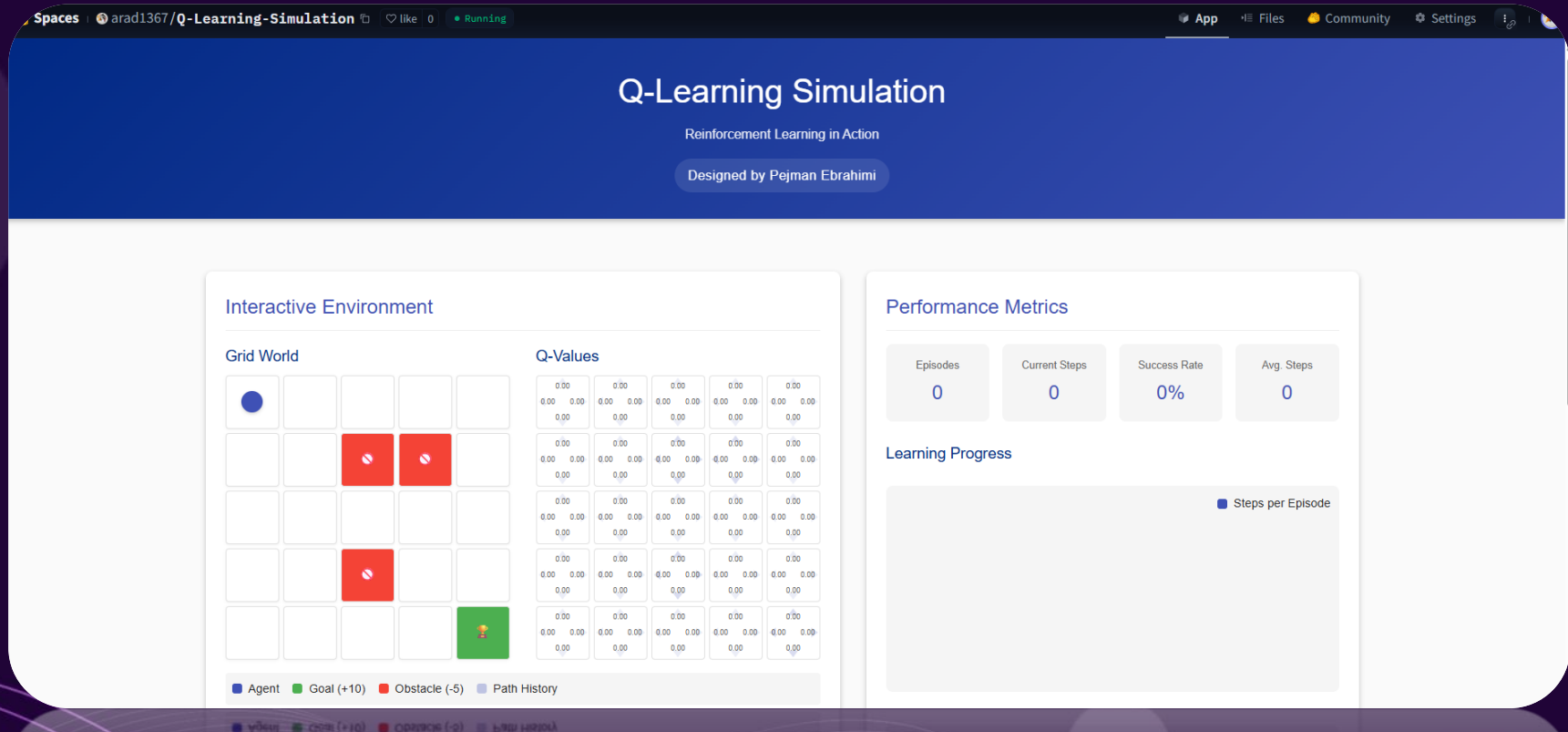
Q-Table

	0	1	2	3
0	→ (1.8)	→ (3.1)	→ (4.6)	↓ (6.2)
1	↑ (0.6)	X	X	↓ (8.0)
2	↓ (3.1)	X	→ (8.0)	↓ (10.0)
3	→ (5.8)	→ (8.0)	→ (10.0)	GOAL

Learning Progress



Q-Learning Simulation



More about Gamma (Discount factor) & Alpha (Learning rate)

Gamma as the discount factor:

- If $\text{GAMMA} = 1$: Future rewards would be valued equally to immediate rewards. The agent would care just as much about rewards far in the future as it does about immediate rewards.
- If $\text{GAMMA} = 0$: The agent would only care about immediate rewards and ignore future rewards completely.
- With $\text{GAMMA} = 0.99$: Future rewards are valued at 99% of their actual value. This means the agent cares a lot about future rewards, but slightly less than immediate rewards.

ALPHA as the learning rate:

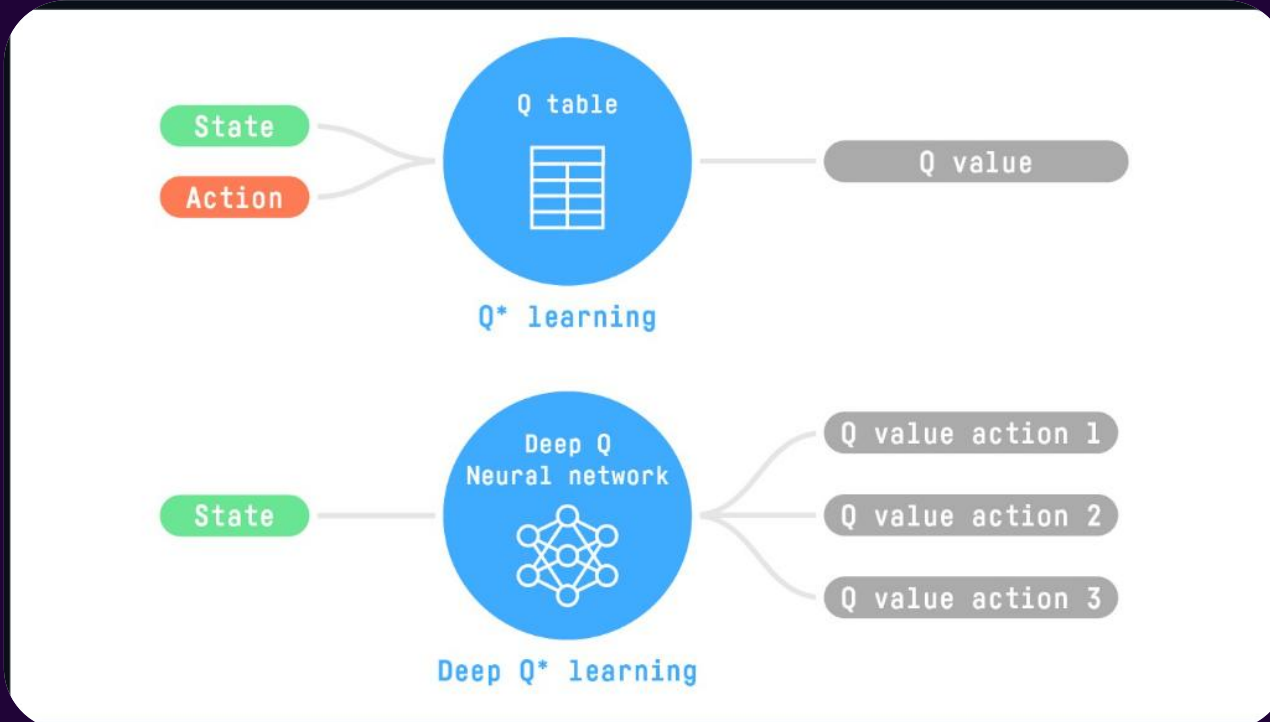
- If $\text{ALPHA} = 1$: The old Q-value would be completely replaced by the new estimate. This would result in very rapid learning, but also high volatility and potential instability.
- If $\text{ALPHA} = 0$: The Q-value would never update, and no learning would occur.
- With $\text{ALPHA} = 0.3$: The Q-value is updated by moving 30% of the way toward the new estimate.
 - If the new estimate is higher than the old Q-value, the Q-value will increase, but only by 30% of the difference.
 - If the new estimate is lower than the old Q-value, the Q-value will decrease, but only by 30% of the difference.

Deep Q-Learning

- Deep Q-Learning. Instead of using a Q-table, Deep Q-Learning uses a Neural Network that takes a state and approximates Q-values for each action based on that state.



Q-Learning & Deep Q-Learning



Q-Learning & Deep Q-Learning

🌟 Q-learning (Top Half of the Image):

- Uses a Q-table. This table stores Q-values for every state-action pair.
- It works well when the number of states and actions is small.
- Think of it like a big Excel sheet with numbers.

🤖 Deep Q-learning (Bottom Half of the Image):

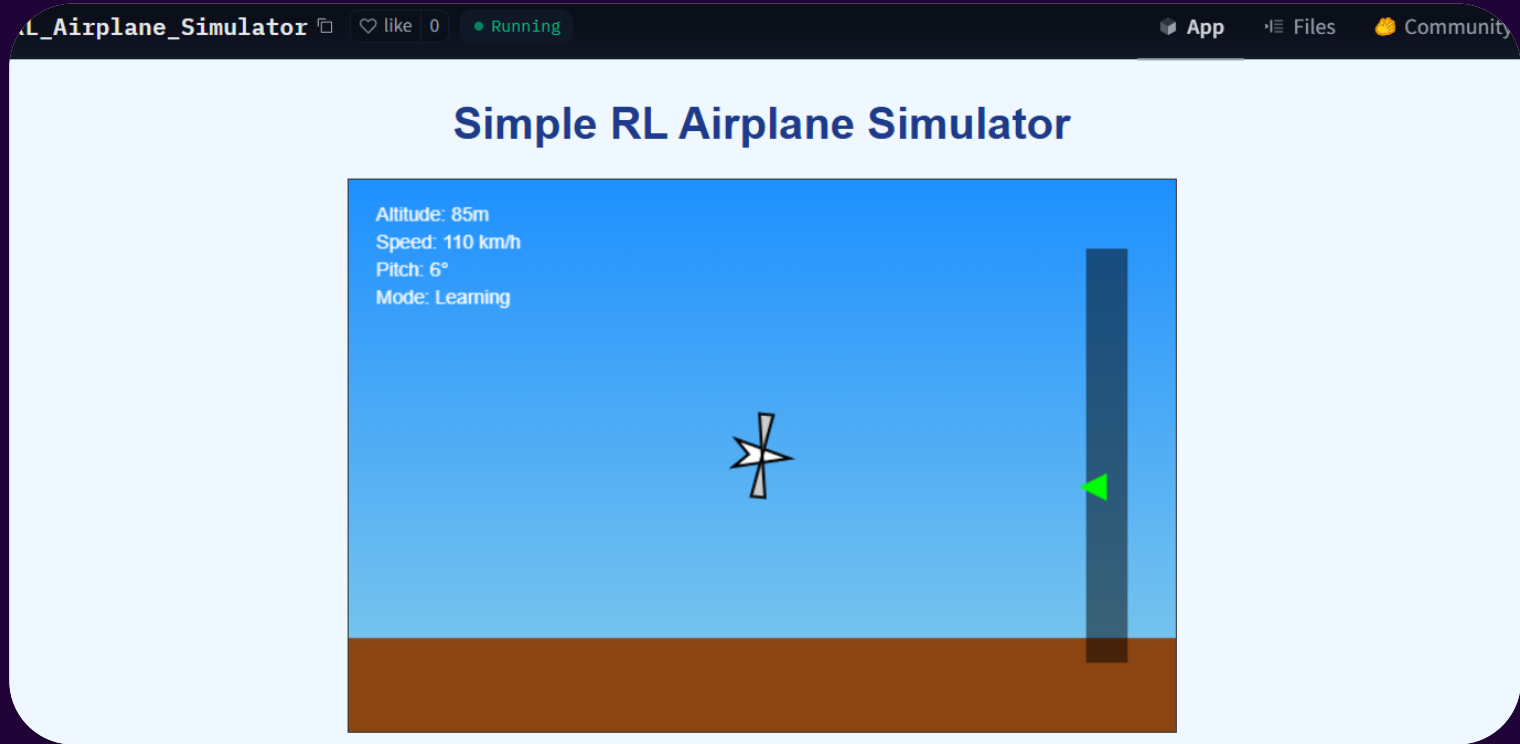
- Uses a neural network instead of a table. The input is just the state.
- The network gives Q-values for all actions at once.
- This is better for large or continuous state spaces where a table would be too big to handle.



03

RL Games

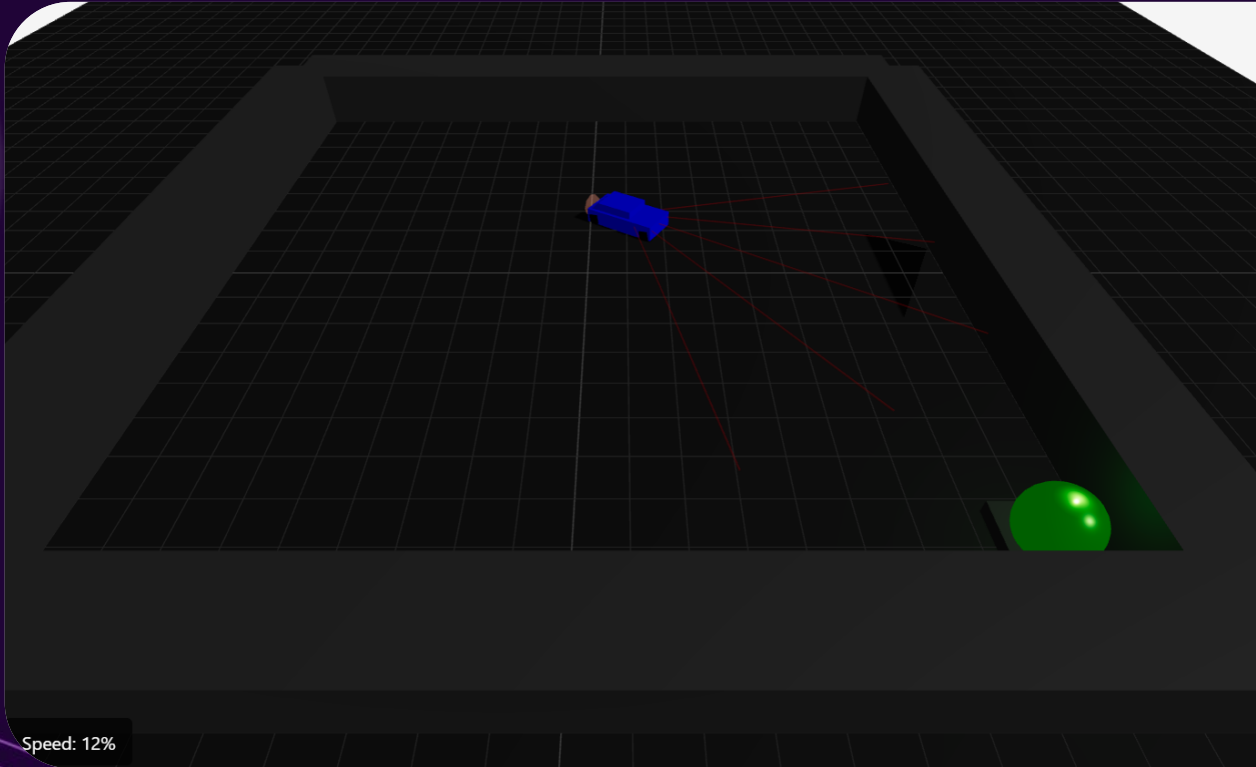
Flight Simulator By Pejman



Huggy By Thomas Simonini



Self-Driving Car Reinforcement Learning Simulation



Speed: 12%

Reinforcement Learning Parameters

Learning Rate: 0.100

How quickly the agent learns from new experiences.

Higher values: Learn faster but may be unstable
Lower values: Learn more slowly but more consistently

Gamma (Discount Factor): 0.90

How much the agent values future rewards vs. immediate ones.

Higher values: More focus on long-term goals
Lower values: More focus on immediate rewards

Episodes: 10000

Number of training runs to perform.

Higher values: More thorough learning
Lower values: Faster training overall

☒ AI Mode



04

Proximal Policy Optimization (PPO)

value-based method

Main idea:

They learn how *good* it is to be in a state (or to take an action in a state). The policy (i.e., how to act) is **indirectly** derived from this knowledge.

- **Focus:** Estimate a **value function** like $Q(s, a)$ or $V(s)$
- **How it works:**
You learn values for actions and then pick the one with the highest value (greedy or ϵ -greedy)
- **Example:** Q-learning, Deep Q-Networks (DQN)

Analogy:

Imagine you're choosing a restaurant. You assign a score (value) to each one and then go to the one with the highest score. You're not directly learning *how* to pick, just the *scores*.

highest score. You're not directly learning how to pick, just the scores.

imagining you're choosing a restaurant, you assign a score (value) to each one and then go to the one with the highest score.

policy-based method

Main idea:

They learn the **policy directly** — that is, they learn the mapping from state to action **without** using a value function.

- **Focus:** Learn a **policy** $\pi(a|s)$, which tells you the probability of taking action a in state s
- **How it works:**
Use optimization methods (like gradient ascent) to improve the policy based on performance
- **Example:** REINFORCE, Proximal Policy Optimization (PPO)

Analogy:

Back to the restaurant example: instead of assigning scores, you just get better at *directly choosing* a good restaurant every time, based on past success.

restaurant every time, based on past success.

back to the restaurant example: instead of assigning scores, you just get better at directly choosing a good

Policy Loss

1. Policy Loss (in PPO)

- ❑ The **policy network** decides which action to take.
 - ❑ **Policy loss** is how much worse or better the new policy is compared to the old one.
 - ❑ It uses something called a "**clipped objective**" to avoid big jumps in the policy.
 - ❑ **Goal:** Improve the agent's behavior without changing it too much in one step.
-  Think of this as:
- "Let's make the agent choose better actions, but not change its mind too drastically."

Value Loss

 **2. Value Loss (in PPO)** PPO also trains a value network, which predicts the value of a state — how much total reward you can expect from there.

Value loss measures how different the predicted value is from the actual return the agent experienced.

 **Think of this as: “Let’s teach the agent to better estimate how good each situation is.”**

Key Concept in PPO

Policy optimization - The agent learns a policy (which action to take in each state) by adjusting probabilities for moving up, down, left, or right

Value function estimation - The agent learns to predict future rewards from each state

Clipped objective function - A key innovation in PPO that prevents too large policy updates

Multiple epochs of updates - PPO reuses experience data for multiple learning updates

What's special about PPO?

PPO solves major problems with earlier reinforcement learning methods:

Stability - The "clipped objective" prevents destructive large policy updates

Sample efficiency - Multiple training epochs on the same data

Performance - Consistently good results across many environments

Simplicity - Easier to implement than many other advanced RL algorithms

Feature	Value-Based Method	Policy-Based Method
Learns	Value function	Policy directly
Policy derivation	Indirect (from values)	Direct
Examples	Q-learning, DQN	REINFORCE, PPO
Stochastic policies	Harder to handle	Easier to represent
Best for	Simple actions	Continuous or complex actions

Why PPO?

Idea

The idea with Proximal Policy Optimization (PPO) is that we want to improve the training stability of the policy by limiting the change you make to the policy at each training epoch: we want to avoid having too large of a policy update. **Measuring how much the new policy differs from the old one Using a special formula (called a "clipped objective") to keep updates small and safe**

Reasons

We know empirically that smaller policy updates during training are more likely to converge to an optimal solution. A too-big step in a policy update can result in falling "off the cliff" (getting a bad policy) and taking a long time or even having no possibility to recover.

Term

So with PPO, we update the policy conservatively. To do so, we need to measure how much the current policy changed compared to the former one using a ratio calculation between the current and former policy. And we clip this ratio in a range $[1 - \epsilon, 1 + \epsilon]$ $[1-\epsilon, 1+\epsilon]$, meaning that we remove the incentive for the current policy to go too far from the old one (hence the proximal policy term).

PPO Simulation

Spaces

arad1367/PPO-Simulation

like 0

Running

App

Files

Community

Settings

Proximal Policy Optimization (PPO) Simulation

This simulation demonstrates how an agent learns to navigate to a goal using **Proximal Policy Optimization (PPO)**. PPO is an on-policy reinforcement learning algorithm that uses a "clipping" mechanism to prevent large policy updates, making training more stable and efficient.

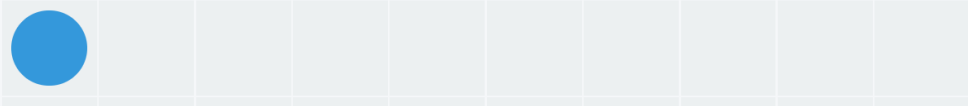
Simulation

PPO Concepts

Training Metrics

Environment

The agent (blue) must navigate to the goal (green) while avoiding obstacles (red).





05

RLHF

Reinforcement learning from human feedback (RLHF)

That is a machine learning (ML) technique that uses human feedback to optimize ML models to self-learn more efficiently. Reinforcement learning (RL) techniques train software to make decisions that maximize rewards, making their outcomes more accurate. RLHF incorporates human feedback in the rewards function, so the ML model can perform tasks more aligned with human goals, wants, and needs. RLHF is used throughout generative artificial intelligence (generative AI) applications, including in large language models (LLM).

Task



Prepare and deliver a 10–15 minutes presentation explaining the concept of Reinforcement Learning with Human Feedback (RLHF) to a peer audience. Your presentation should include:

- ❑ **Definition** – What is RLHF?
- ❑ **Motivation** – Why is human feedback used in reinforcement learning?
- ❑ **Basic Workflow** – Describe the typical steps in an RLHF training process (e.g., pretraining, reward modeling, fine-tuning).
- ❑ **Real-World Applications** – Mention at least one example (e.g., ChatGPT training).
- ❑ **Benefits & Challenges** – Highlight key advantages and current limitations of RLHF.

Thank You

Do you have any questions?

pejman.ebrahimi@uni.li

